



AFSS

BACKLOG TRACKER
SYSTEM WORKFLOW DOCUMENT

6

WORKFLOWS

3

COMPANIES

30m

AUTO-REFRESH

1hr

CACHE TTL

Data Flow

Job Lifecycle

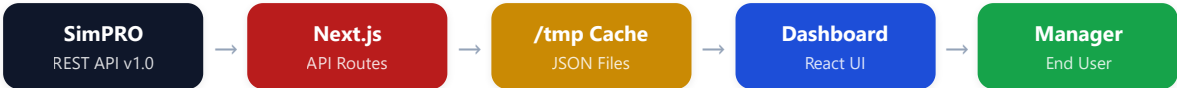
Cache Strategy

Dashboard Loading

Refresh Cycle

Automation

All data originates from SimPRO's REST API and travels through a caching layer before reaching the dashboard. This diagram shows how each layer connects end-to-end.



SimPRO API Calls

- STEP 1 — JOB LIST

GET /companies/{id}/jobs/

Pages 1–10 fetched in parallel. Filtered to CFSP tech ID 1126. Returns up to 2,500 jobs per company/stage.
- STEP 2 — JOB DETAILS (PER JOB, CONCURRENCY 10)

GET /companies/{id}/jobs/{jobId}

Full job object — labour hours, status name, all enrichment fields.
- STEP 3 — SCHEDULES (PER JOB, CONCURRENCY 10)

GET /companies/{id}/schedules/?JobID={id}

Confirmed booking date and total scheduled hours per job.
- STEP 4 — SITES (PER UNIQUE SITE, CONCURRENCY 10)

GET /companies/{id}/sites/{siteId}

Site name and address. Deduplicated — each unique site fetched once.
- STEP 5 — CUSTOMERS (PER UNIQUE CUSTOMER, CONCURRENCY 10)

GET /companies/{id}/customers/{segment}/{id}

Customer profile and group name. Deduplicated across all jobs.

RATE LIMITING

Steps 2–5 run in parallel streams, up to 40 concurrent API calls. SimPRO enforces HTTP 429 limits — each call retries up to 8× with exponential backoff (1s → 2s → 4s → 8s...).

API Routes → Dashboard

Route	Feeds	TTL
/api/data	Tech Team Works table (all status rows)	1 hr
/api/leave	Technical Team Supply (hours + leave note)	1 hr
/api/tech-support	Technical Support Works (all 3 columns)	1 hr
/api/warmup	Pre-warms all three caches above	—

Dashboard Fetch Pattern

The dashboard fires **7 parallel fetches** on load:

PARALLEL REQUESTS ON PAGE LOAD

▶ Company 1 Pending — RM AFSS pending jobs

▶ Company 1 Progress — RM AFSS in-progress jobs

▶ Company 8 Pending — CHUBB/AFAC pending jobs

▶ Company 8 Progress — CHUBB/AFAC in-progress jobs

▶ Company 10 Pending — AE Evac pending jobs

▶ Company 10 Progress — AE Evac in-progress jobs

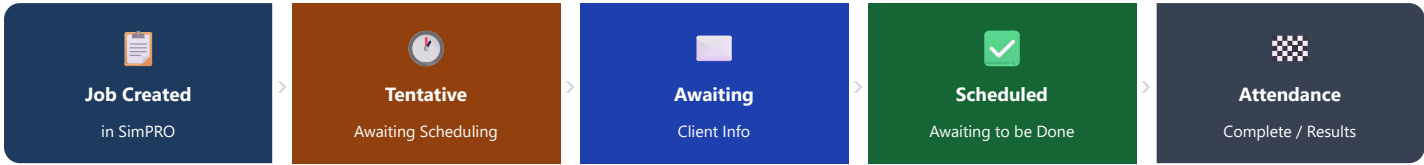
▶ /api/leave — Team leave for current month

Each response renders immediately as it arrives — the table updates company-by-company rather than waiting for all 7. /api/tech-support loads independently in the bottom section.

CLOUD SCHEDULER

Google Cloud Scheduler hits /api/warmup every 30 minutes automatically, keeping caches populated before users ever request data.

Every CFSP audit job moves through a series of statuses on its way from creation to completion. The dashboard reflects each stage in a different colour-coded row.



How Dashboard Classifies Each Job

- 1 Check: Is Stage = Progress? (non-AE Evac)**
→ Attendance Complete / Results To Be Released
Stage = Progress means attendance has occurred for RM AFSS and CHUBB/AFAC companies.
- 2 Check: Is this AE Evac (Company 10) + Stage = Progress?**
AE Evac uses Progress to mean "booking confirmed." Classify as Scheduled if a date exists, or Tentative if no date.
- 3 Check: Does the job have a confirmed scheduled date?**
Date detected from SimPRO schedule blocks or job detail fields (Scheduled, DateScheduled, etc.).
→ Scheduled Awaiting to be Done
- 4 Check: Does Status Name include "awaiting" or "client"?**
→ Awaiting Client Info
Job is pending but blocked on the client side.
- 5 Default — no date, no client block**
→ Tentative Awaiting Scheduling
Job exists but has not been booked or awaited on client.

Status Rules Per Company

Company	Stage	Has Date?	Dashboard Row
RM AFSS (1)	Pending	Yes	Scheduled
RM AFSS (1)	Pending	No	Tentative
RM AFSS (1)	Progress	Any	Complete
CHUBB (8)	Pending	Yes	Scheduled
CHUBB (8)	Pending	No	Tentative
CHUBB (8)	Progress	Any	Complete
AE Evac (10)	Pending	Yes	Scheduled
AE Evac (10)	Pending	No	Tentative
AE Evac (10)	Progress	Yes	Scheduled
AE Evac (10)	Progress	No	Tentative

CFSP Job Filter

Only jobs with technician ID 1126 ("A CFSP ONLY") assigned appear on the dashboard — except Company 8 (CHUBB/AFAC) which shows all jobs.

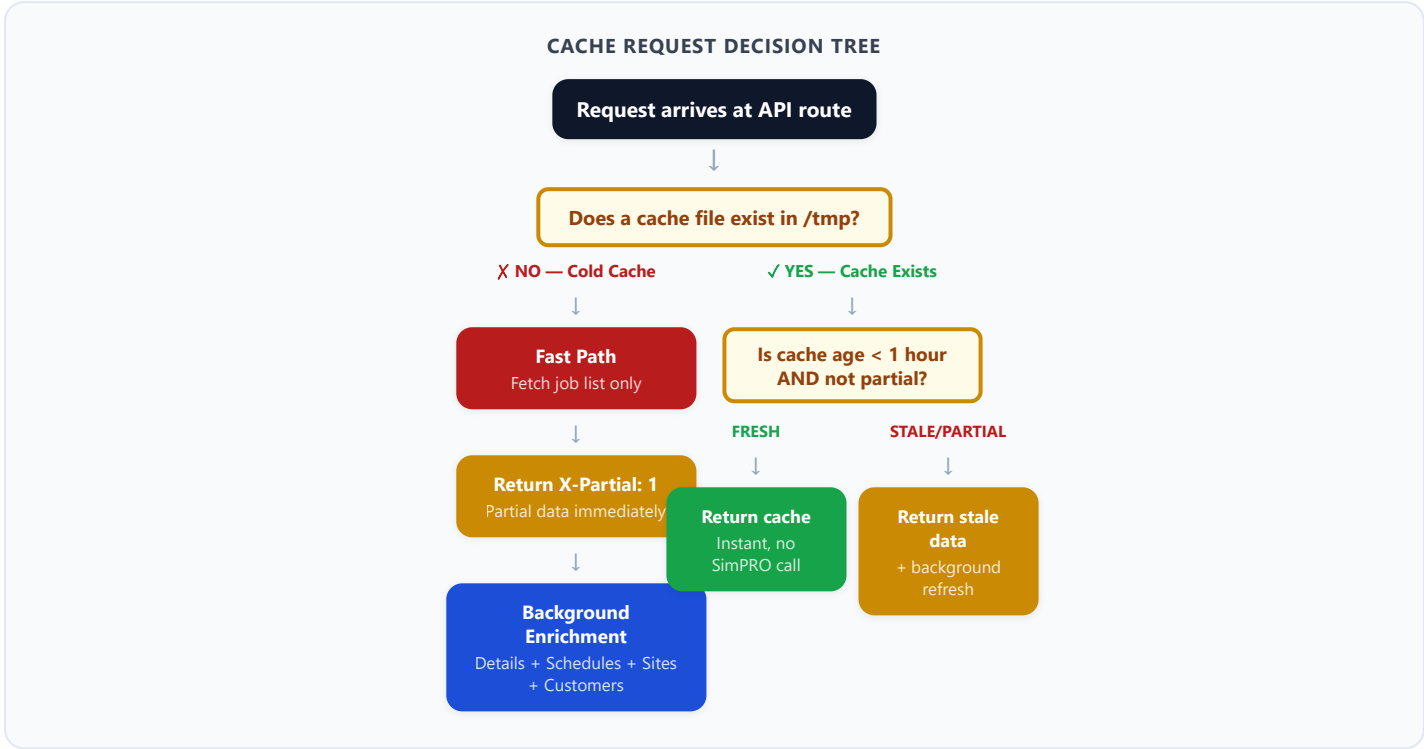
MONTH FILTER EFFECT

When a month is selected (top-right dropdown), only jobs whose **scheduled date** falls in that month are counted in status rows. The Total Backlog row always shows all jobs regardless of filter.

MIXED PLAN TYPE EXCLUSION

If a Completed job has another "A ..." plan-type technician alongside "A CFSP ONLY" (e.g. "A Hydrant"), it is excluded from the Complete row to avoid double-counting across dashboards.

Every API route follows the same caching lifecycle. Understanding this flow explains why the dashboard is fast under normal conditions and what happens on a cold start.



Cache State Descriptions

State	Condition	User Experience
Fresh	Age < 1 hr, fully enriched	Instant response, correct numbers
Stale	Age ≥ 1 hr, fully enriched	Instant response (old data), background refresh starts
Partial	Enrichment not yet complete	Fast response, — dashes in Scheduled/Tentative, auto-retries every 5 s
Cold	No cache file	Fast-path data in ~1–2 s, correct numbers in 30–160 s

COLD CACHE TRIGGER

A cold cache occurs when Cloud Run **recycles the container** — new deployment, idle scale-down, or instance restart wipes `/tmp`. The Cloud Scheduler warmup job prevents users from ever hitting cold cache under normal operation.

Cache Files on Disk

File	Contents	TTL
afss-jobs-1-Pending.json	RM AFSS Pending jobs	1 hr
afss-jobs-1-Progress.json	RM AFSS Progress jobs	1 hr
afss-jobs-8-Pending.json	CHUBB/AFAC Pending jobs	1 hr
afss-jobs-8-Progress.json	CHUBB/AFAC Progress jobs	1 hr
afss-jobs-10-Pending.json	AE Evac Pending jobs	1 hr
afss-jobs-10-Progress.json	AE Evac Progress jobs	1 hr
afss-leave-cache.json	Team leave schedule	1 hr
afss-tech-support-v18-cache.json	Tech Support stats	1 hr

PARTIAL FLAG

Each cache file stores a `partial: true/false` flag alongside the data. A fresh but partial cache means enrichment was incomplete on the last write — it's still served immediately but triggers background re-enrichment.

Dashboard Loading Workflow

The dashboard is designed to show something useful as fast as possible, then progressively improve the data until it is fully accurate — without ever blocking the user on a blank screen.

Normal Load (Warm Cache)

- **0 MS — USER OPENS /DASHBOARD**
7 parallel fetches fire immediately
 6 company/stage + leave. "Loading..." indicator appears in nav bar.
 Scheduled & Tentative rows show — dashes (isPartial=true set upfront).
- **~200–800 MS — CACHE HIT RESPONSES ARRIVE**
Table populates company by company
 Each company's data renders the moment its fetch returns. No waiting for all 6 to complete. Job counts, hours, amounts all visible.
- **~800 MS — ALL 6 RESPONSES IN**
isPartial=false — full data confirmed
 "Loading..." disappears. Scheduled and Tentative rows show correct numbers. "Updated: HH:MM:SS" timestamp appears.
- **~1 S — BOTTOM SECTION LOADS**
Leave + Tech Support Works appear
 Team supply hours and Technical Support Works section populate independently from the main table.

TOTAL TIME — WARM CACHE

Dashboard is fully loaded and accurate in **under 2 seconds**. This is the normal experience when the Cloud Scheduler warmup has run within the last 30 minutes.

Cold Cache Load Sequence

- **0 MS — USER OPENS /DASHBOARD**
7 parallel fetches fire
 isPartial set to true immediately — Scheduled & Tentative rows show — dashes from the start. No wrong numbers flicker.
- **~1–3 S — FAST-PATH RESPONSES ARRIVE**
Partial data with X-Partial: 1 header
 Job list data (count, total hours, amounts) appears. Scheduled & Tentative show — because enrichment hasn't completed. Amber "Refreshing schedule data..." shows.
- **EVERY 5 SECONDS**
Dashboard auto-retries all 6 fetches
 partialTimerRef fires every 5 s. Each retry checks if the background enrichment has written new cache. Serves whatever is available immediately.
- **30–160 S — BACKGROUND ENRICHMENT COMPLETES**
Full cache written to /tmp
 Next 5-second retry gets enriched data — X-Partial header absent. isPartial set to false. Correct Scheduled / Tentative split appears. Amber indicator disappears.
- **~35–165 S — DASHBOARD FULLY ACCURATE**
All rows show correct numbers
 Auto-retry stops. All subsequent users get instant warm-cache responses. Cold start recovers automatically.

Refresh Cycle & Automation Workflow

30m

AUTO-WARMUP INTERVAL

1hr

CACHE TTL

5s

PARTIAL RETRY INTERVAL

310s

WARMUP MAX DURATION

Workflow 5 — Automatic Cache Warmup

1 Cloud Scheduler fires every 30 minutes

Job: `afss-warmup-job` — schedule `* / 30 * * * *` UTC. Sends GET request to `/api/warmup`.

2 warmup route runs all three warmers in parallel

`warmAll()` + `warmLeave()` + `warmTechSupport()` run simultaneously.

3 warmAll() processes 6 combos sequentially

Company 1 Pending → Company 1 Progress → Company 8 Pending → Company 8 Progress → Company 10 Pending → Company 10 Progress. Sequential to avoid SimPRO 429 rate limits.

4 Each combo runs full fetchAndCache()

Job list → 4 parallel enrichment streams → write to `/tmp/afss-jobs-{co}-{stage}.json` with `partial: false`.

5 Warmup returns success report

Returns `{"ok": true, "results": [{company, stage, ok, ms}, ...]}`. All subsequent user requests served from fresh cache instantly.

WHY SEQUENTIAL FOR JOBS?

Running all 6 company/stage combos in parallel would fire 240+ simultaneous SimPRO API calls, causing severe rate limiting (many 429s) and actually increasing total time. Sequential keeps it to ~40 concurrent calls at a time.

Workflow 6 — Manual Refresh Now Button

1 User clicks "Refresh now"

Dashboard calls `load(force=true)`, `loadLeave(force=true)`, and `loadTechSupport(force=true)` simultaneously.

2 isPartial set to true immediately

Scheduled & Tentative rows show — dashes instantly. No stale numbers briefly visible.

3 force=1 requests hit the API

API skips cache check. Runs fast path: fetches job list immediately, returns `X-Partial: 1` response within ~1–2 s. Background enrichment kicks off.

4 Dashboard updates progressively

Job counts appear company-by-company as each fast-path response arrives. Table is live within 2 s.

5 Auto-retry until enrichment completes

5-second retry loop runs until `X-Partial` header is gone. Scheduled/Tentative rows show — until then, then snap to correct numbers.

Post-Deployment Warmup

Every new deployment creates a fresh container with empty `/tmp`. The following command must be run after each deploy to pre-populate all caches before users arrive:

```
# PowerShell — run after gcloud deploy
Invoke-WebRequest `
  -Uri "https://bryan-technical-afss-712513641417
    .australia-southeast1.run.app/api/warmup" `
  -TimeoutSec 310
```

WARMUP DURATION

Full warmup takes **2–5 minutes** depending on SimPRO API speed. Company 1 Pending (85 jobs) and Company 10 Progress (123 jobs) are the slowest, taking 45–160 s each. All other combos complete in under 20 s.